Lecture 5: Expectations — Transition Dynamics

第五讲：预期 — 过渡动态

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 15, 2026

University of Toronto

Course Overview

Where we are
我们走到哪里了

- L1–L3: build the household block from **Stages**.
- L4: close the model with an **outer loop**; solve Aiyagari SS.
- **Today (L5):** meet the package; replicate L4; introduce MIT shocks.

- L1–L3：用 **Stage** 搭家庭模块。
- L4：用**外层循环**闭合模型；求 Aiyagari 稳态。
- **今天（L5）：**见包、复现 L4、引入 MIT 冲击。

Today's roadmap
今日路线图

1. What is `HouseholdStages`?
2. Replicate the Aiyagari SS using it.
3. MIT shock: a new dynamic experiment.
4. Solving an MIT transition (abstract).
5. Implementation + IRFs + diagnostics.

1. 什么是 `HouseholdStages`?
2. 用它复现 Aiyagari 稳态。
3. MIT 冲击：新的动态实验。
4. 求解 MIT 过渡（抽象层面）。
5. 实现 + IRF + 诊断。

What is HouseholdStages?

Prior art (a quick note)
已有工作（小注）

- Stage-based household models aren't new.
- Auclert–Bardóczy–Rognlie–Straub's **SSJ toolkit** (Python) ships its own stage implementations.
- This course's `HouseholdStages.jl` is a Julia counterpart, focused on the V/Λ duality and per-stage functorial lifts.

- Stage 化的家庭模型并不新鲜。
- Auclert–Bardóczy–Rognlie–Straub 的 **SSJ toolkit**（Python）已有 Stage 实现。
- 本课程的 `HouseholdStages.jl` 是 Julia 版，着重 V/Λ 对偶与单 Stage 函子提升。

Recall L4's promise
回顾第四讲的承诺

L4 told you:

- Stages have precise math.
- Stages are standardized.
- You can download optimized implementations.

Today: that package is `HouseholdStages.jl`. From here on you call the library — same math, same V/Λ duality.

第四讲讲过：

- Stage 有精确数学定义。
- Stage 是标准化的。
- 优化实现可以下载。

今天：那个包就是 `HouseholdStages.jl`。从今天起就调库——数学不变、V/Λ 对偶不变。

What the package gives you
包给了你什么

- **Stage primitives.** `MarkovStage`, `WealthChangeStage`, `ConsumptionSavingsStage`, ...
- **Composition.** $\circ_s$ chains Stages; `lift_moments` attaches integrals.
- **Inner-solve helpers.** V, Λ at a given env.

- **Stage 原语。** `MarkovStage`、`WealthChangeStage`、`ConsumptionSavingsStage` 等。
- **组合。** $\circ_s$ 串 Stage；`lift_moments` 挂积分。
- **内层求解器。** 给定 env 下的 V 与 Λ。

Also in the library (not today)
库里还有（今天不讲）

- **More Stages.** `LogitChoiceStage`, `MigrationStage`, `BorrowingConstraintStage`, `AssetPriceChangeStage`.
- **Functorial lifts.** `lift_jacobian` (auto-diff), `lift_gpu`.
- **SSJ utilities.** Sequence-space Jacobian (covered in L6+).

- **更多 Stage。** `LogitChoiceStage`、`MigrationStage`、`BorrowingConstraintStage`、`AssetPriceChangeStage`。
- **函子提升。** `lift_jacobian`（自动微分）、`lift_gpu`。
- **SSJ 工具。** 序列空间 Jacobian（L6+ 讲）。

What the package does *not* do
包不做的事

The library handles **the household block at a given env**. It does *not*:

- Decide your aggregate state.
- Compute prices from $\bar{K}$ (firm block).
- Run the outer Walrasian tatonnement.
- Define a transition path.
- Pick a calibration.

库管的是**给定 env 下的家庭模块**。它不：

- 决定你的总量状态。
- 由 $\bar{K}$ 算价格（厂商部分）。
- 跑外层 Walras tatonnement。
- 定义过渡路径。
- 选校准。

Why this split
为什么这样切

- Household block: **generic** across HA models.
- Outer loop, calibration, transition: **model-specific**.
- Library boundary at "*given env, solve the household problem*."
- Everything else is yours to write.

- 家庭模块：跨 HA 模型**通用**。
- 外层、校准、过渡：**模型相关**。
- 库的边界划在" 给定 *env*，解家庭问题"。
- 其余都你自己写。

The three inner-solve helpers
三个内层求解器

```
# V backward to a fixed point at fixed env.
solve_vfi_steady_state_given_env!(hh, env, buffers)

# Λ forward to stationarity using whatever policy backward populated.
solve_lambda_steady_state_given_env!(hh, buffers)

# Bundle of the two: V then Λ.
solve_steady_state_given_env!(hh, env, buffers)
```

Principle. All three take env for granted. They iterate the household block alone — no outer loop, no aggregate-state update.

原则。三者都把 env 当已知，只迭代家庭模块本身——不动外层、不更新总量状态。

Aiyagari SS, redone

Same model as L4
和第四讲同一个模型

- **Chain** (time order):

  $\text{IncomeShock} \circ_s \text{IncomeReceipt} \circ_s \text{ConsumptionSavingsStage}$

- **Firm.** Cobb-Douglas, fixed labor L.
- **Equilibrium.** $\bar{K} = \bar{K}^{\text{supplied}}(\bar{K})$.
- **Outer loop.** Damped **tatonnement** on $\bar{K}$.

- **链**（时间顺序）：

  $\text{IncomeShock} \circ_s \text{IncomeReceipt} \circ_s \text{Consumption}$

- **厂商。** Cobb-Douglas，劳动 L 给定。
- **均衡。** $\bar{K} = \bar{K}^{\text{supplied}}(\bar{K})$。
- **外层。** 对 $\bar{K}$ 做阻尼 **tatonnement**。

Why tatonnement, not bisection
为什么不用二分

- L4 used bisection on $\bar{K}$.
- In section 5 the unknown becomes a *path* $\{K_t\}$ — T-dimensional.
- Bisection has no T-dim analog (no scalar to bracket).
- Tatonnement does: just update K_t pointwise.
- We adopt tatonnement here too, for continuity.

- 第四讲对 $\bar{K}$ 用二分。
- 第 5 节未知变成路径 $\{K_t\}$ ——T 维。
- 二分在 T 维下没有类比（没有标量可二分）。
- Tatonnement 行：对 K_t 逐期更新。
- 这里也用 tatonnement，保持一致。

State layout
状态空间

```
layout = StateLayout(
    StateAxis(:wealth, continuous_grid(0, 100; length=400, spacing=:log)),
    StateAxis(:income, [0.6, 1.0, 1.4]),
)
```

- **wealth** (continuous, 400-pt log grid).
- **income** (discrete, 3 levels).
- Closures read `cell.wealth`, `cell.income`.

- **wealth**（连续、400 点对数格点）。
- **income**（离散、3 个水平）。
- 闭包可读 `cell.wealth`、`cell.income`。

Stage 1: IncomeShock
第 1 阶段：收入冲击

```
income_shock = MarkovStage(layout; axis = :income, transition = P_y)
```

- L4: hand-rolled `income_shock_backward(V, p) = V * p.Π'`.
- Package: one line.
- K-operator is the transition matrix; backward $= K^\top$, forward $= K$.

- 第四讲手写 `income_shock_backward(V, p) = V * p.Π'`。
- 包里：一行。
- K 算子就是转移矩阵；后向 $K^\top$，前向 K。

Stage 2: IncomeReceipt
第 2 阶段：领工资

```
income_receipt = WealthChangeStage(layout;
    wealth_post = (cell; env) ->
        (1 + env.r) * cell.wealth + env.w * cell.income,
    wealth_axis = :wealth,
)
```

- L4: snapped $Rb + wy$ to the nearest grid index.
- Package: linear V interpolation backward; share-based Λ redistribution forward.

- 第四讲：把 $Rb + wy$ 落到最近格点。
- 包里：后向线性 V 插值；前向份额式 Λ 再分配。

Stage 3: ConsumptionSavingsStage
第 3 阶段：消费–储蓄

```
savings = ConsumptionSavingsStage(layout;
      = 0.96, utility = (cell, c; env) -> c < 0 ? -Inf : log(c),
    wealth_axis = :wealth, monotone_search = :divide_conquer)
```

- L4: built $N \times N$ trial matrix, took `maximum`.
- Package: picks b_{end} on the grid; $c = b - b_{\text{end}}$ implied.
- `:divide_conquer`: $O(N \log N)$ when policy is monotone in b.

- 第四讲：建 $N \times N$ 试矩阵，取 `maximum`。
- 包里：在格点上选 b_{end}，$c = b - b_{\text{end}}$。
- `:divide_conquer`：策略对 b 单调时 $O(N \log N)$。

Compose; lift a moment
组合；提升一个矩

```
hh = lift_moments(income_shock    income_receipt    savings;
    K_supplied = at_end(integrand = :wealth, reduce = sum),
)
```

- $\circ_s$ chains Stages in time order.
- The household block is *itself a Stage*: own `backward!` and `forward!`.
- `lift_moments`: attach integrals against terminal Λ. Here $K^{\text{supplied}} = \int b \, \mathrm{d}\Lambda$.

- $\circ_s$ 按时间顺序串 Stage。
- 整条链本身就是 *Stage*：自带 `backward!` 和 `forward!`。
- `lift_moments`：把积分挂到链尾 Λ 上。这里 $K^{\text{supplied}} = \int b \, \mathrm{d}\Lambda$。

Inner solve at a single $\bar{K}$
给定 $\bar{K}$ 的内层解

```
env  = (; K, aiyagari_prices(K, p)...)          # (K, r, w)
info = solve_steady_state_given_env!(hh, env, buffers; V_init = V, Λ_init = Λ)
K_supplied = compute_moments(hh, env).K_supplied
```

- **Line 1.** Build env: (K, r, w).
- **Line 2.** V backward; Λ forward.
- **Line 3.** Read off K^{supplied}.

- **第 1 行。**造 env：(K, r, w)。
- **第 2 行。**V 后向；Λ 前向。
- **第 3 行。**读 K^{supplied}。

Outer tatonnement loop
外层 tatonnement 循环

```
K = 5.0
while iters < max_iter
    env  = (; K, aiyagari_prices(K, p)...)
    info = solve_steady_state_given_env!(hh, env, buffers; V_init = V, Λ_init = Λ)
    V, Λ = info.V, info.Λ
    K_supplied = compute_moments(hh, env).K_supplied
    K_err      = abs(K_supplied - K) / K
    K_err <= rtol && break
    K += update_speed * (K_supplied - K)        # damped tatonnement
end
```

Each pass: inner solve $\to$ read K^{supplied} $\to$ nudge $\bar{K}$. Stop on $|K_{\text{err}}|/K <$ `rtol`.

每轮：内层解 $\to$ 读 K^{supplied} $\to$ 调 $\bar{K}$。$|K_{\text{err}}|/K <$ `rtol` 即停。

Run it
跑一下

```
iter 17: K = 5.6921 → K_supplied = 5.5446, K_err = 0.025925
iter 18: K = 5.6847 → K_supplied = 5.6731, K_err = 0.002055
Converged in 18 outer iterations.
  K   = 5.6847,  r   = 0.0384,  w   = 1.1964
```

- Same model as L4. Shorter code path.
- L4 landed $\bar{K} \approx 5.30$ with coarser grid + bisection.
- We land $\bar{K} = 5.68$ with 400-pt exp grid.

- 和第四讲同一模型，代码更短。
- 第四讲粗格点 + 二分 $\bar{K} \approx 5.30$。
- 这里 400 点指数格点 $\bar{K} = 5.68$。

Comparative statics

One SS at a time
一个稳态一个稳态地算

- Same household block, same algorithm.
- Change one parameter (here: A). Re-solve.
- No transition, no time path. Just *two* steady states.

Concretely:

- $A = 1 \to \bar{K}_{\text{pre}} = 5.68$, $\bar{R}_{\text{pre}} = 0.038$.
- $A = 1.05 \to \bar{K}_{\text{post}} = 6.14$, $\bar{R}_{\text{post}} = 0.038$.

- 同一个家庭模块、同一个算法。
- 调一个参数（这里：A），重算稳态。
- 没有过渡、没有时间路径。只是两个稳态。

具体地：

- $A = 1 \to \bar{K}_{\text{pre}} = 5.68$，$\bar{R}_{\text{pre}} = 0.038$。
- $A = 1.05 \to \bar{K}_{\text{post}} = 6.14$，$\bar{R}_{\text{post}} = 0.038$。

What comparative statics tells you
比较静态能告诉你什么

- The *destination*: where the economy ends up.
- Sign and magnitude of long-run effects.
- **What it doesn't tell you:** how fast, what path, who is hurt during transit.

The new $\bar{K}_{\text{post}}$ is computed *without reference to* any time path: same tatonnement, just a different A.

- 终点：经济最终走到哪。
- 长期效应的方向与幅度。
- **它不告诉你：**多快、走什么路径、过渡期里谁受损。

新的 $\bar{K}_{\text{post}}$ 是不参照任何时间路径算出来的：同一个 tatonnement，换个 A 就行。

Need transition dynamics to answer
要回答这些问题，得做过渡

- How long does adjustment take?
- Does K_t overshoot before settling at $\bar{K}_{\text{post}}$?
- Welfare gain on impact vs in the long run.
- Who saves vs dissaves during transit.

These are *path* questions. The rest of L05 is about answering them.

- 调整要多长时间?
- K_t 会不会冲过 $\bar{K}_{\text{post}}$ 再回落?
- 即期 vs 长期的福利变化。
- 过渡期里谁在存、谁在花。

这些都是路径问题。L05 后半节讲怎么算它们。

MIT shock

What's a transition path?
什么是过渡路径?

- **Steady state.** A single (K, V, Λ). Time-invariant.
- **Transition.** A sequence $\{(K_t, V_t, \Lambda_t)\}_{t=1,\dots,T}$. The economy is *moving*.

Why we care:

- Welfare during an unexpected reform.
- Speed of convergence to a new SS.
- Matching IRFs in calibration.

- **稳态。**单点 (K, V, Λ)，与时间无关。
- **过渡。**序列 $\{(K_t, V_t, \Lambda_t)\}_{t=1,\dots,T}$。经济在动。

为什么关心：

- 未预期改革下的福利分布。
- 收敛到新稳态的速度。
- 校准中匹配 IRF。

The MIT shock
MIT 冲击

Standardized by Boppart–Krusell–Mitman (2018).

Three ingredients:

- $t = 0$: at steady state.
- $t = 1$: one-time impulse. Agents *surprised*.
- $t \geq 1$: *perfect foresight* of the entire future path.

Boppart–Krusell–Mitman 2018 把它做成了标配。

三个要素：

- $t = 0$：处于稳态。
- $t = 1$：一次性冲击，主体毫无预期。
- $t \geq 1$：完美预期整条未来路径。

Why use it
为什么用它

- Name from MIT macro courses. Teaching device.
- Tradeoff: unrealistic, but tractable.
- No expectations integrals. No aggregate randomness.
- Deterministic version of a stochastic IRF.

- 名字来自 MIT 宏观课堂。教学设计。
- 权衡：不现实，但可解。
- 没有期望积分，没有总量随机性。
- 随机 IRF 的确定性版本。

Our example shock
我们的示例冲击

Setup. *Permanent* positive TFP step at $t = 1$:

$$A_t = 1.05 \quad \text{for all } t \geq 1, \quad A_0 = 1.$$

- The *only* exogenous driver.
- Unanticipated at $t = 0$; known forever after.
- Everything else — $\{r_t, w_t, K_t, V_t, \Lambda_t\}$ — is the **response**.
- Pick $T = 100$ so we reach the new long-run steady state.

设置。 $t = 1$ 起 TFP 永久上升：

$$A_t = 1.05 \quad \text{对所有} t \geq 1, \quad A_0 = 1.$$

- 唯一的外生驱动。
- $t = 0$ 时未预期；之后全程已知。
- 其它都是**响应**：$\{r_t, w_t, K_t, V_t, \Lambda_t\}$。
- 取 $T = 100$，足以走到新的长期稳态。

The TFP path in code
TFP 路径的代码

```
# Permanent step: A_t = A_0 for all t >= 1.
tfp_path(T; A_0 = 1.05) = fill(A_0, T)
```

Defined locally. The library leaves transition utilities to the consumer — nothing in `HouseholdStages` touches anything beyond a single env.

本地定义。库把过渡相关的工具一律交给调用方——`HouseholdStages` 不碰单个 env 之外的任何东西。

Solving an MIT transition

The problem statement
问题陈述

Given: $\{A_t\}_{t=1}^{T}$, the household block, the firm.

Find: $\{K_t, V_t, \Lambda_t\}$ such that

- $V_{T+1} = V_{\text{ss}}$ (terminal).
- $\Lambda_1 = \Lambda_{\text{ss}}$ (initial).
- $V_t =$ household block's backward at (K_t, A_t).
- $\Lambda_{t+1} =$ household block's forward on Λ_t.
- Markets clear: $K_t = K_t^{\text{supplied}}$.

已知： $\{A_t\}_{t=1}^{T}$、家庭模块、厂商。

求： $\{K_t, V_t, \Lambda_t\}$ 满足

- $V_{T+1} = V_{\text{ss}}$（终端）。
- $\Lambda_1 = \Lambda_{\text{ss}}$（初始）。
- $V_t =$ 链在 (K_t, A_t) 上的后向。
- $\Lambda_{t+1} =$ 链对 Λ_t 的前向。
- 市场出清：$K_t = K_t^{\text{supplied}}$。

The four steps
四个步骤

1. Guess a path $\{R_t\}$. Pin K_t from R_t via the firm FOC.

2. Backward sweep on V.

$$V_{T+1} \leftarrow V_{\text{ss}^{\text{post}}}, \quad V_t = \texttt{backward!}(V_{t+1}, \text{env}_t).$$

3. Forward sweep on Λ.

$$\Lambda_1 \leftarrow \Lambda_{\text{ss}^{\text{pre}}}, \quad \Lambda_{t+1} = \texttt{forward!}(\Lambda_t).$$

4. Tatonnement on R_{path}.

$$R_t \leftarrow (1-d)\, R_t + d\, R_t^{\text{implied}}.$$

1. 猜一个路径 $\{R_t\}$。由企业一阶条件钉住 K_t。

2. V 后向扫。

$$V_{T+1} \leftarrow V_{\text{ss}^{\text{post}}}, \quad V_t = \texttt{backward!}(V_{t+1}, \text{env}_t).$$

3. Λ 前向扫。

$$\Lambda_1 \leftarrow \Lambda_{\text{ss}^{\text{pre}}}, \quad \Lambda_{t+1} = \texttt{forward!}(\Lambda_t).$$

4. 对 R_{path} tatonnement。

$$R_t \leftarrow (1-d)\, R_t + d\, R_t^{\text{implied}}.$$

Why backward for V
为什么 V 要后向

- $V_t(s)$ = value at t, given everything from t onward.
- **Time-indexed Bellman:**

$$V_t(s) = \max_a \; u_t(s,a) + \beta\, \mathbb{E}\, V_{t+1}(s'(s,a)),$$

  argmax policy $\pi_t(s)$.
- Value flows future $\rightarrow$ present: V_t depends on V_{t+1}.
- Terminal $V_{T+1} = V_{\text{ss}^{\text{post}}}$; walk backward.

- $V_t(s)$ = t 时的价值，已知 t 起一切。
- **带时间下标的 Bellman：**

$$V_t(s) = \max_a \big\{ u_t(s,a) + \beta\, \mathbb{E}\, V_{t+1}(s'(s,a)) \big\},$$

  最优策略 $\pi_t(s)$。
- 价值从未来流向当下：V_t 依赖 V_{t+1}。
- 从已知 $V_{T+1} = V_{\text{ss}^{\text{post}}}$ 起，倒着走。

Why forward for Λ
为什么 Λ 要前向

- $\Lambda_t =$ mass distribution over states at t.
- Mass flows forward: tomorrow comes from today.
- Forward step: $\Lambda_{t+1} = K\Lambda_t$.
- Start from $\Lambda_1 = \Lambda_{\text{ss}}$; walk forward.
- Initial condition: shock at $t = 1$ is the first SS departure.

- $\Lambda_t = t$ 期主体在各状态上的质量分布。
- 质量沿时间前流：明天由今天决定。
- 前向步：$\Lambda_{t+1} = K\Lambda_t$。
- 从 $\Lambda_1 = \Lambda_{\text{ss}}$ 前推。
- 初始条件：$t = 1$ 冲击是首次偏离稳态。

Tatonnement on a sequence
在序列上做 tatonnement

Scalar (L4 / section 2):

$$K \leftarrow K + \mathrm{lr} \cdot (K^{\text{supplied}} - K).$$

Path (L5): pointwise.

$$K_t \leftarrow (1 - d)\, K_t + d\, K_t^{\text{supplied}}.$$

- Newton with the sequence-space Jacobian (next lecture) is faster.
- Same loop structure.

标量（L4 / 第 2 节）：

$$K \leftarrow K + \mathrm{lr} \cdot (K^{\text{supplied}} - K).$$

路径（L5）：逐期。

$$K_t \leftarrow (1 - d)\, K_t + d\, K_t^{\text{supplied}}.$$

- 下讲序列空间 Jacobian + Newton 更快。
- 循环结构一样。

The algorithm
算法

```
K_t = fill(K_ss, T)                    # initial guess
V_path[T+1] = V_ss; Λ_path[1] = Λ_ss  # boundary conditions
while res > tol
    for t in T:-1:1                                  # backward sweep
        V_path[t] = backward!(hh, V_path[t+1], env_t)
    end
    for t in 1:T                                     # forward sweep
        backward!(hh, V_path[t+1], env_t)            #   re-seat policy
        Λ_path[t+1]   = forward!(hh, Λ_path[t])
        K_supplied[t] = compute_moments(hh, env_t).K_supplied
    end
    res = maximum(abs, K_supplied .- K_t)            # damped tatonnement
    K_t .= (1 - damping) .* K_t .+ damping .* K_supplied
end
```

Two `for`-loops in a `while`-loop. 两个 `for` 嵌在 `while` 里。

Implementation

Warm start from the steady state
用稳态做热启动

We need both:

- V_{ss}: terminal for backward sweep.
- Λ_{ss}: initial for forward sweep.

Section 2's tatonnement gives both.

Setup: $V_{\text{ss}} \to V_{\text{path}}[T+1]$;
$\Lambda_{\text{ss}} \to \Lambda_{\text{path}}[1]$; initial $\{K_t\} \equiv K_{\text{ss}}$.

两个都要：

- V_{ss}：后向扫终端条件。
- Λ_{ss}：前向扫初始条件。

第 2 节的 tatonnement 同时给出二者。

设置： $V_{\text{ss}} \to V_{\text{path}}[T+1]$；
$\Lambda_{\text{ss}} \to \Lambda_{\text{path}}[1]$；$\{K_t\}$ 初值 $\equiv K_{\text{ss}}$。

Backward sweep
后向扫描

```
for t in T:-1:1
    env_t = (; K = K_t[t], aiyagari_prices(K_t[t], A_path[t], p)...)
    V_path[t] .= backward!(hh, V_path[t+1], env_t, buffers)
end
```

- T calls to `backward!`, $t = T \to 1$.
- Each mutates per-stage `kernels` (kernels, policies).
- After the loop, kernels hold period 1's policy.

- 调 T 次 `backward!`，$t = T \to 1$。
- 每次都会改写各 Stage 的 `kernels`（kernel、策略）。
- 循环后缓存停在第 1 期。

Forward sweep (re-seat kernels!)
前向扫描（缓存要重设!）

```
for t in 1:T
    env_t = (; K = K_t[t], aiyagari_prices(K_t[t], A_path[t], p)...)
    backward!(hh, V_path[t+1], env_t, buffers)   # re-seat
    Λ_path[t+1]   .= forward!(hh, Λ_path[t], buffers)
    K_supplied[t]  = compute_moments(hh, env_t).K_supplied
end
```

Why the second `backward!`**?** Forward reads policy from `kernels`; policy depends on the env. The backward sweep left kernels at period 1, so we re-seat at period t first.

为什么还要 `backward!`**?** 前向从 `kernels` 读策略，策略依赖该期 env。后向扫已把缓存挪到第 1 期，要先重设到第 t 期。

Outer tatonnement update
外层 tatonnement 更新

```
res = maximum(abs, K_supplied .- K_t)
K_t .= (1 - damping) .* K_t .+ damping .* K_supplied
```

- Vectorized: every period gets the same damped Walrasian update.
- Residual: ∞-norm over the path.
- Below `tol`: path consistent everywhere.

- 向量化：每期同样阻尼的 Walras 更新。
- 残差：路径上的 ∞-范数。
- 降到 `tol` 以下：路径处处一致。

Run it
跑起来

```
Computing steady state…
  K_ss = 5.6847, r_ss = 0.0384, w_ss = 1.1964

Solving transition (T = 100, A_0 = 1.05,   = 0.85, d = 0.2)…
  iter  5: ‖K_supplied - K‖∞ = 0.0639
  iter 17: ‖K_supplied - K‖∞ = 0.0018   ← discretization floor

  K[1] = 5.74    K[5] = 5.87    K[20] = 5.81    K[100] = 5.69
```

End-of-period: $K[1] = 5.74$ already — agents save *within* period 1. Peak at $t \approx 5$. Residual floors at $\sim 2.5 \times 10^{-3}$ (diagnostics next).

期末口径：$K[1] = 5.74$ ——第 1 期内已开始储蓄。峰值在 $t \approx 5$。残差停在 $\sim 2.5 \times 10^{-3}$（下面诊断）。

IRF: capital
脉冲响应：资本

Shape.

- ΔA raises r.
- Higher $r \to$ more savings.
- More saving $\to$ higher $K \to$ lower r. Self-correcting.

Lag. Wealth distribution has inertia. Peak in K lags peak in A by ~ 5 periods.

形状。

- ΔA 抬升 r。
- 更高的 $r \to$ 更多储蓄。
- 更多储蓄 $\to$ 更高 $K \to$ 回落 r。自我修正。

时滞。财富分布有惯性。K 峰落在 A 峰后约 5 期。

IRF: r_t and w_t
脉冲响应：r_t 与 w_t

r_t.

- Jumps on impact (A_1 before K moves).
- Falls below SS as K overshoots.
- Returns to r_{ss} as $A_t \to 1$.

w_t.

- Same pattern, opposite sign.
- Above SS while K overshoots.

r_t。

- 第 1 期冲上（A_1 早于 K 反应）。
- K 过冲让 r 跌到稳态之下。
- $A_t \to 1$ 时回到 r_{ss}。

w_t。

- 形状相同，方向相反。
- K 过冲期间高于稳态。

Welfare
福利

- Wealthy agents: transient capital gain on r.
- Wage-dependent agents: pick up the w side.

For Aiyagari we get a *distribution* of welfare effects, period by period. Disentangling the two channels is one thing MIT shocks let you do.

- 富人：在 r 上拿到暂时性资本收益。
- 靠工资的人：从 w 一侧获益。

Aiyagari 给出福利效应的分布，逐期看。把两条腿拆开是 MIT 冲击的用处之一。

Pick T large enough
T 要够大

- $T =$ horizon where $A_T \approx 1$ and $V_{T+1} = V_{\text{ss}}$.
- Too small $\rightarrow$ terminal condition contaminates the path.

Rule of thumb. T such that $A_T - 1 < 10^{-3}$. At $\rho = 0.85$, $A_0 = 1.05$: $T \approx 25$. We use $T = 100$.

- $T =$ 假设 $A_T \approx 1$、$V_{T+1} = V_{\text{ss}}$ 的时窗。
- 太小 $\rightarrow$ 终端条件污染整条路径。

经验法则。取 T 使 $A_T - 1 < 10^{-3}$。$\rho = 0.85$、$A_0 = 1.05$ 时约 $T = 25$。这里取 $T = 100$。

Cost of the algorithm
算法成本

- Per outer iteration: $O(T)$ inner ops.
- Each inner op $\approx$ one L4 inner Bellman step.
- ~ 20 outer iterations $\times$ $T = 100 \rightarrow$ $\sim 2000\times$ a single L4 step.

Manageable. SSJ + Newton (next lecture) cuts this dramatically.

- 每次外层迭代：$O(T)$ 次内层操作。
- 每次内层 $\approx$ 第四讲一次 Bellman 步。
- 约 20 次外层 $\times$ $T = 100 \rightarrow$ 约第四讲单步的 2000 倍。

可控。下讲 SSJ + Newton 会大幅压缩。

Damping is a craft
阻尼是个手艺

- **Too high.** Oscillation. Residual stalls or grows.
- **Too low.** Slow. Residual decays at near-1 rate.
- **Sweet spot here.** $d \in [0.15, 0.25]$; we use 0.2.

Problem-dependent. Bigger / more persistent shocks $\rightarrow$ lower d.

- **太大。**振荡。残差停滞或增大。
- **太小。**爬得慢。残差以接近 1 的比率下降。
- **这里的最佳。**$d \in [0.15, 0.25]$，我们取 0.2。

依赖问题。冲击越大、越持续 $\rightarrow$ d 越小。

Residual history
残差历史

What you should see. Geometric decay, then a *discretization floor*.

Why a floor.

- `ConsumptionSavingsStage` picks b_{end} via `argmax` on a discrete grid.
- Near the borrowing constraint, optimal b_{end} flips between adjacent cells as K_t wobbles.

Fix. Smoothed `LogitChoiceStage` savings, or refine the grid.

应当看到。几何下降，然后撞上离散化下限。

为什么有下限。

- `ConsumptionSavingsStage` 在离散格点上用 `argmax` 选 b_{end}。
- 借贷约束附近，K_t 微动让最优 b_{end} 在相邻格子间翻转。

对策。用平滑的 `LogitChoiceStage`，或加密格点。

Wrap

What we built today
今天搭出了什么

- Met `HouseholdStages.jl`.
- Replicated the Aiyagari SS using it.
- Switched outer loop: bisection → tatonnement.
- Introduced MIT shocks.
- Solved a deterministic perfect-foresight transition.

- 见到了 `HouseholdStages.jl`。
- 用它复现了 Aiyagari 稳态。
- 外层：二分 → tatonnement。
- 引入了 MIT 冲击。
- 解了一条确定性完美预期过渡。

The key insight
关键洞察

The household block doesn't care SS vs. transition.

- Same household block. Same V/Λ duality. Same library calls.

Only the outer loop changes:

- SS: tatonnement on a scalar.
- Transition: tatonnement on a path.

家庭模块不在乎是稳态还是过渡。

- 同一条链。同一个 V/Λ 对偶。同一组库调用。

变的只有外层：

- 稳态：在标量上 tatonnement。
- 过渡：在路径上 tatonnement。